# Assignment 2: Paper Summarization and Reproduction

HyperImpute: Generalized Iterative Imputation with Automatic Model Selection
Jarrett, Cebere, Liu, Curth, van der Schaar (ICML 2022)

Yann Klees 0800619
Data Wrangling, Utrecht University

24 October 2026

## Part 1. Paper summary

### 1. Main problem

Existing imputation methods fall into two families, and each has a strength the other lacks. Iterative imputers such as MICE [3] and MissForest [4] fit a flexible model to every column. They also work on incomplete data when values are missing at random (MAR), so that missingness depends only on observed values. The catch is that each column model has to be chosen and tuned by hand. Deep generative imputers such as GAIN [5] and MIWAE [6] need no manual tuning. They are hard to train, however, and most need complete training data or assume that values are missing completely at random (MCAR). HyperImpute [1] aims to combine flexible models per column, automatic configuration and training on incomplete data under MAR.

### 2. Proposed solution

HyperImpute keeps the column by column structure of iterative imputation. The difference is that the user no longer fixes one model in advance. For each column, the method searches over several model classes and their hyperparameters (Section 3.2). Algorithm 1 (Section 3.4) has four steps.

1. Fill every missing cell with a simple first guess, by default the column mean.
2. Visit the columns one at a time. Train a model on the rows where the column is observed, with all other columns, including their current guesses, as inputs.
3. Try several learners and hyperparameters, keep the one that predicts the column best, and use it to fill the column's missing rows.
4. Repeat over all columns until the imputations stop changing.

The candidate pool holds linear and logistic regression, random forests, XGBoost [7], CatBoost [8] and small neural networks. Continuous columns get regressors and categorical columns get classifiers (Section 4, Appendix A.3). If the search is switched off and each column gets a fixed model, the algorithm becomes classic iterative imputation. HyperImpute therefore generalises it.

### 3. Optimization techniques and special cases

Section 3.3 argues that the key design choice is where the model search sits relative to the imputation loop. A top down search reruns the whole imputation for every combination of column models. With $D$ columns and $A$ configurations each, that means $A^D$ runs, which is far too expensive. A concurrent search tunes each column on its own at a cost of $O(KDA)$ for $K$ rounds. It ignores, however, how one column's model affects the others. HyperImpute turns this around. The imputation loop runs on the outside, and a small search for each column runs inside it. The cost stays at $O(KDA)$, yet every search sees the latest imputations of all other columns.

By default, the search is an adapted Hyperband [9] (Section 4, Appendix A.4). Hyperband trains many candidates briefly, drops the weak ones and gives the survivors more training. Neural networks count training in epochs and forests in trees, so both are mapped onto one common budget. Naive search and Bayesian optimisation are also available. Later rounds can skip the search and reuse earlier choices. The loop stops after a maximum number of rounds, when the imputations barely change, or when the objective stops improving (Appendix A.5).

## 4. Strengths and limitations

**Strengths.**

- It removes the manual model choice that holds iterative imputation back, and it inherits the properties of iterative imputation under MAR.
- Unlike most deep generative imputers, it trains on incomplete data and handles mixed data types (Table 1).
- An ablation (Table 2) credits a performance gain to each design component.

**Limitations.**

- Nothing guarantees that the separate column models form a valid joint distribution over all variables.
- It returns a single imputation, so it does not quantify how uncertain the filled values are.
- MNAR lies outside the paper's scope, and the paper gives no theory for it.

## 5. Evaluation methodology

The authors remove values artificially from 12 UCI datasets [10], so the true values are known. They simulate MCAR, MAR and MNAR at rates from 10% to 70%, with MAR at 30% as the main setting. HyperImpute is compared with eight baselines, among them MICE, MissForest, GAIN and MIWAE. Errors are measured with RMSE and the Wasserstein distance (WD), averaged over 10 seeds.

HyperImpute is best on both metrics on 10 of the 12 datasets and wins at least one metric on every dataset. In the ablation, it lowers RMSE by 18% on average compared with the best single learner variant. The search picks simple models on small datasets and at high missingness, and boosting and neural networks on large ones. The imputations settle after about four rounds.

# Part 2. Reproduction

Code: `https://github.com/YannKleees/hyperreimpute`. I use the authors' `hyperimpute` package [2], version 0.1.17, with the settings of their notebook `experiments_04_gain_of_function.ipynb`. As a check, I also use version 0.1.5, which was released with the paper. All runs were on CPU.

```
git clone https://github.com/YannKleees/hyperreimpute.git && cd hyperreimpute
pip install -r requirements.txt
python -m hyperreimpute.experiments    # writes results/*.csv
python -m hyperreimpute.version_check  # here and in venv_015, see README
```

## 1. Selected experiment

I reproduce the source of gains ablation (Table 2 of [1]). It tests the central claim that choosing a model for each column improves iterative imputation. As in the paper, 30% of the values are removed under MAR, with 10 seeds. I use airfoil, compression and white wine, since the other two datasets were too slow on a laptop. Two extra controls use a properly configured CatBoost.

## 2. Comparison with the paper

**Table 1.** RMSE × 10 on the masked cells under MAR with 30% missingness, averaged over 10 seeds. Lower is better.

| | airfoil | | compression | | white wine | |
|---|---|---|---|---|---|---|
| Method | Paper | Ours | Paper | Ours | Paper | Ours |
| ice_lr | 2.35 | 1.95 | 1.43 | 1.49 | 0.92 | 0.86 |
| ice_rf | 2.37 | 1.91 | 1.62 | 1.68 | 0.99 | 0.90 |
| ice_cb | 2.08 | 2.55 | 1.25 | 2.29 | 0.88 | 1.10 |
| global_search | 1.60 | 1.92 | 1.03 | 1.45 | 0.85 | 0.79 |
| column_naive | 1.86 | 1.32 | 1.09 | 1.14 | 0.85 | 0.79 |
| wo_flexibility_cb | 1.59 | 2.16 | 1.08 | 2.00 | 0.88 | 1.10 |
| wo_adaptivity | 1.66 | 1.92 | 1.05 | 1.58 | 0.89 | 0.79 |
| HyperImpute | 1.48 | 1.80 | 1.01 | 1.48 | 0.80 | 0.79 |
| ice_cb_fixed | n/a | **1.25** | n/a | **1.04** | n/a | **0.75** |
| column_naive_cb_fixed | n/a | **1.25** | n/a | **1.04** | n/a | **0.75** |

Row names follow the paper. The ice rows use one learner for all columns, column_naive picks one untuned model per column, and the rows in between switch off parts of HyperImpute. The paper's results vary a lot between seeds, so I compare against the uncertainty of its means and test differences between methods with paired Wilcoxon signed rank tests [11] over the seeds, which share the same masks.

The simple baselines reproduce only in part. On airfoil, every one of them beats the paper by roughly a fifth to a quarter. The shift always has the same sign, which suggests different masking. On compression the baselines match the paper, and on white wine they are slightly better.

Choosing a model per column clearly beats a single linear model or random forest, by about a third on airfoil, a quarter on compression and a tenth on white wine, on every seed. Against the best single learner, though, it loses. A properly configured CatBoost beats both per column selection and full HyperImpute on all three datasets. When this CatBoost joins the pool, per column selection simply picks it and gives practically the same results. Selection thus finds a good learner when the pool has one, so its gain depends on how weak the other learners are. The paper's gain over the best single learner does not reproduce here.

Full HyperImpute reproduces only on white wine. On airfoil and compression its error is a fifth to a half higher than reported, and its Wasserstein distance is almost three times as high. Tuning even makes it about a third worse than per column selection without tuning, and stretches a run from under a minute to several minutes. The other tuned variants of the paper fall behind the untuned version on these two datasets too, while the paper reports the opposite.

## 3. Parameters

All runs use `hyperimpute` 0.1.17 (0.1.5 for the check), pandas below 2.3 and Python 3.11 on CPU. Data are min max scaled and masked under MAR at 30% with `column_limit=10` and seeds 0 to 9 (0 to 4 in the check). RMSE uses the masked cells; the Wasserstein distance is summed over columns.

**Table 2.** Settings.

| Method | Optimizer | Column | Iteration | Lazy | Learners |
|---|---|---|---|---|---|
| HyperImpute | hyperband | yes | yes | no | full pool |
| global_search | hyperband | no | no | yes | full pool |
| wo_adaptivity | hyperband | yes | no | yes | full pool |
| wo_flexibility_cb | hyperband | no | no | yes | CatBoost only |
| column_naive | simple | yes | no | yes | full pool |
| ICE variants | simple | no | no | yes | one fixed learner |
| ice_cb_fixed | simple | no | no | yes | CatBoost, 1000 trees, learning rate 0.03 |
| column_naive_cb_fixed | simple | yes | no | yes | full pool with this CatBoost |

The full pool holds linear models, random forest, XGBoost and CatBoost; neural networks need a GPU and were not used. Imputation starts from the column mean and visits columns in random order. The search uses `n_inner_iter=40`, `select_patience=5` and `optimize_thresh=5000` (50, 3 and 1000 in 0.1.5). Other settings keep the defaults.

## 4. Reasons for the differences

The most likely cause is that the hyperimpute package changed after the paper came out. The current version gives CatBoost only 10 trees and a very small learning rate by default. Its predictions therefore barely move away from the column mean, which explains the weak CatBoost baseline and the CatBoost only variant. The version released with the paper left the learning rate to CatBoost itself, and the paper's list of CatBoost settings (Table 5) has none either. Later versions also changed the search ranges of CatBoost and XGBoost, the default size of the random forest and the number of iterations.

To test this, I reran HyperImpute, per column selection and the CatBoost baseline with version 0.1.5 on airfoil and compression, with five seeds and the same masks. The CatBoost baseline improves strongly and gets close to the paper on compression. HyperImpute improves on every seed, by about a fifth, and its Wasserstein distance now matches the paper. It still trails per column selection slightly, and that method barely changes. The package version therefore explains the CatBoost results and the gap in Wasserstein distance. It does not explain why tuning fails.

## 5. Conclusion

Choosing a model per column clearly beats a single linear model or random forest, but not a well configured CatBoost, and full HyperImpute matches the paper only on white wine. The paper's package version explains the CatBoost results, but not why tuning fails to help. Reproductions should fix the package version and check every learner's settings.

## References


[1] D. Jarrett et al. HyperImpute: Generalized iterative imputation with automatic model selection. *ICML*, 2022.

[2] HyperImpute source code. `https://github.com/vanderschaarlab/hyperimpute`

[3] S. van Buuren, K. Groothuis-Oudshoorn. mice: Multivariate imputation by chained equations in R. *J. Stat. Softw.* 45(3), 2011.

[4] D. J. Stekhoven, P. Bühlmann. MissForest: Nonparametric missing value imputation for mixed-type data. *Bioinformatics* 28(1), 2012.

[5] J. Yoon, J. Jordon, M. van der Schaar. GAIN: Missing data imputation using generative adversarial nets. *ICML*, 2018.

[6] P.-A. Mattei, J. Frellsen. MIWAE: Deep generative modelling and imputation of incomplete data sets. *ICML*, 2019.

[7] T. Chen, C. Guestrin. XGBoost: A scalable tree boosting system. *KDD*, 2016.

[8] L. Prokhorenkova et al. CatBoost: Unbiased boosting with categorical features. *NeurIPS*, 2018.

[9] L. Li et al. Hyperband: A novel bandit-based approach to hyperparameter optimization. *JMLR* 18, 2018.

[10] D. Dua, C. Graff. UCI Machine Learning Repository, 2019.

[11] F. Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin* 1(6), 1945.